# Trabalho Prático 05 — Agentes Inteligentes para o Jogo Othello

GCC 128 — Inteligência Artificial · Prof. Ahmed Ali Abdalla Esmin

Vinícius Arantes e Mariana Rufo

Novembro de 2026

## 1. Introdução

Este trabalho implementa um **agente inteligente que joga Othello** (Reversi) com uma LLM inserida num fluxo de decisão estruturado com **LangGraph**. A cada turno o agente **percebe** o ambiente, **consulta ferramentas** (*tools*), **analisa** alternativas, **decide**, **executa** a jogada no próprio ambiente e **registra** o que fez na memória. O ciclo se repete até o fim da partida. A LLM não recebe o tabuleiro para simplesmente devolver uma posição: ela escolhe quais ferramentas chamar, interpreta os resultados e age por meio de `make_move`. Quando propõe uma jogada inválida, recebe o erro do ambiente e corrige.

Para medir o papel de cada componente foram implementados cinco jogadores:

- **LLM Planejador**: o agente principal (seção 5.4);
- **LLM Básico**: laço ReAct com apenas as quatro ferramentas obrigatórias;
- **Minimax (d4)**: busca Alfa-Beta, sem LLM;
- **Heurístico**: avaliação gulosa de um lance, sem LLM;
- **Aleatório**: linha de base.

**Tecnologias.** Python 3.12, LangChain (`langchain-core` 1.8), LangGraph 1.2, Groq (`openai/gpt-oss-120b` e `openai/gpt-oss-20b`, plano gratuito, via `langchain-groq`), Pygame 2.6, pandas/matplotlib e pytest.

## 2. O ambiente: regras do Othello

O motor (`othello/game.py`) representa o tabuleiro com **bitboards**: cada cor é um inteiro de 64 bits, e o bit `8·linha + coluna` indica uma peça. Para gerar as jogadas, o conjunto de peças é deslocado nas 8 direções e acumula sequências de peças adversárias que terminam numa casa vazia. Máscaras impedem que um deslocamento “dê a volta” entre as colunas *a* e *h*. A mesma técnica calcula as peças invertidas por um lance. Por isso a busca Minimax é rápida mesmo em Python: cerca de 0,02 s por lance com profundidade 4.

**Regras implementadas:**

- tabuleiro 8×8 com a posição inicial padrão;
- jogada válida somente se cercar ao menos uma linha de peças adversárias;
- inversão de todas as linhas cercadas;
- **passe obrigatório**, automático, quando o jogador da vez não tem jogada;
- **fim de jogo** quando nenhum dos dois pode jogar, com vencedor pela contagem de peças.

As coordenadas seguem a notação padrão (coluna *a*–*h*, linha *1*–*8*, ex.: `d3`), a mesma do eOthello.

**Validação.** São 11 testes automatizados (`pytest`):

- capturas nas 8 direções e ausência de *wrap-around* nas bordas;
- passe automático e fim de jogo;
- **verificação cruzada com uma implementação ingênua** (matriz 8×8) em 300 partidas aleatórias completas, comparando lance a lance as jogadas válidas e as peças invertidas;
- o Alfa-Beta devolve exatamente o mesmo valor que um Minimax sem poda;
- a busca exata de final de jogo está correta.

## 3. Arquitetura

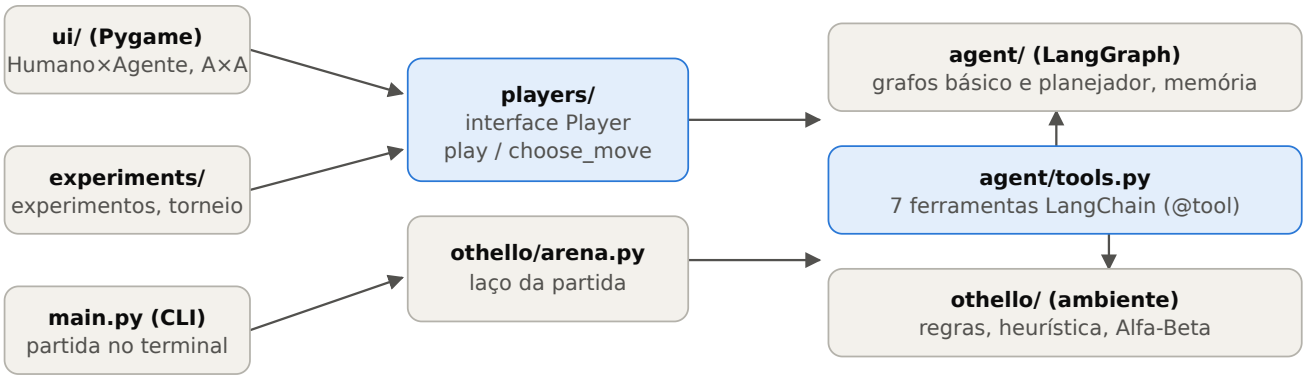



Figura 1 — Módulos do sistema. A UI, os experimentos e a CLI usam a mesma interface de jogador. Os agentes LLM agem sobre o ambiente exclusivamente através das ferramentas.

Todos os jogadores implementam `players.base.Player`. Os jogadores clássicos só precisam de `choose_move(game)`. Os agentes LLM sobrescrevem `play(game)`, porque **são eles que executam a jogada** chamando a ferramenta `make_move` sobre o jogo real. Essa interface única é usada pela interface gráfica, pelos experimentos, pelo torneio e pela CLI, e permite plugar o agente de outro grupo no torneio com um adaptador simples.

## 4. Ferramentas

As ferramentas são funções Python decoradas com `@tool` (LangChain), criadas por `build_tools(env)`. Elas operam sobre um objeto `GameEnv`, que guarda o jogo, a cor do agente, a memória e os contadores de métricas.

| Ferramenta | Tipo | O que retorna / faz |
|---|---|---|
| `get_board_state()` | obrigatória | Tabuleiro em texto (`B`, `W`, `.` e `*` para jogadas válidas), orientação, vez e placar |
| `get_valid_moves()` | obrigatória | Jogadas válidas e quantas peças cada uma vira |
| `make_move(position, reasoning, plan)` | obrigatória | **Executa** a jogada; se for inválida, devolve o erro e a lista de jogadas válidas |
| `get_game_status()` | obrigatória | Em andamento ou encerrada, vez, placar, nº do lance, fase, vencedor |
| `evaluate_move(position)` | análise | Para uma candidata: peças viradas, canto/borda, casa X/C perigosa, se **entrega canto**, mobilidade resultante, score heurístico |
| `search_best_moves(k, depth)` | planejamento | *Top-k* jogadas segundo um Minimax Alfa-Beta (até profundidade 5) |
| `get_move_history(last_n)` | memória | Histórico da partida com as justificativas anteriores |

Toda chamada de ferramenta e de LLM é contabilizada: número de chamadas, latência, tokens de entrada e saída, jogadas inválidas, fallbacks e erros de API. Esses números alimentam os experimentos.

## 5. Estratégia e tomada de decisão

### 5.1 Função de avaliação heurística

`othello/heuristics.py` combina critérios clássicos, com pesos que variam conforme a fase do jogo (abertura > 44 casas vazias; meio-jogo; final ≤ 14):

- **posicional**: matriz de pesos por casa. Cantos valem +100; casas X (b2, g2, b7, g7) e C (vizinhas dos cantos na borda) são negativas **enquanto o canto correspondente está vazio**;
- **mobilidade**: (minhas jogadas − jogadas do oponente) / total;
- **fronteira**: peças vizinhas de casas vazias (quanto menos, melhor);
- **cantos**: posse dos 4 cantos;
- **estabilidade**: peças ancoradas num canto ao longo das bordas, que não podem mais ser viradas;
- **paridade de peças**: só pesa no final.

### 5.2 Minimax com poda Alfa-Beta

`othello/search.py` implementa o Minimax na forma *negamax* com poda Alfa-Beta:

- as jogadas são ordenadas pelo peso posicional (cantos primeiro) para podar mais;
- passes são tratados explicitamente;
- posições terminais valem ±100 000 mais a diferença de peças;
- quando restam 10 casas vazias ou menos, a busca vai até o fim (**solução exata**).

`rank_moves` avalia todas as jogadas com janela completa e alimenta a ferramenta `search_best_moves`.

### 5.3 Agente LLM Básico (ReAct)

É um grafo LangGraph simples, `agent ⇄ tools`, com apenas as quatro ferramentas obrigatórias. A LLM recebe só o histórico de lances da partida e decide sozinha quando observar e quando jogar. Se terminar sem jogar, um nó `nudge` a lembra de chamar `make_move`. Após 6 chamadas sem jogada válida, entra o fallback: uma jogada aleatória, contabilizada nas métricas. Esse agente responde à pergunta "a LLM sozinha, só com acesso ao ambiente, joga bem?".

### 5.4 Agente LLM Planejador (grafo LangGraph)

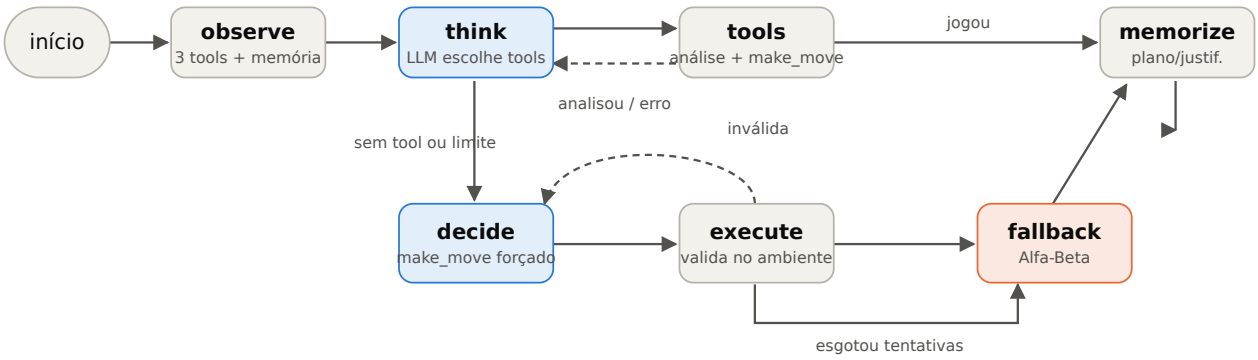



Figura 2 — Grafo do agente Planejador (`agent/graph.py`). Nós em azul chamam a LLM. Linhas tracejadas são laços de realimentação.

O estado do grafo (`PlannerState`) guarda as mensagens do turno, os contadores de rodadas e tentativas e a indicação de fallback. Os nós:

1. **observe (percepção):** chama `get_game_status`, `get_board_state` e `get_valid_moves` e monta a observação com a **memória**: lances da partida, as três últimas decisões do agente com suas justificativas, os últimos lances do oponente e o **plano anterior**.
2. **think (raciocínio):** a LLM recebe princípios estratégicos (cantos, casas X/C, mobilidade, jogadas silenciosas, estabilidade, contagem no final) e escolhe as ferramentas, normalmente `search_best_moves` e/ou `evaluate_move` em paralelo. Ela pode discordar da busca, desde que justifique. Em seguida chama `make_move(position, reasoning, plan)`.
3. **tools / execute (ação):** o `ToolNode` executa as ferramentas. Uma jogada inválida volta como mensagem de erro com as jogadas válidas, e a LLM corrige.
4. **decide (rede de segurança):** se a LLM responder sem chamar ferramenta ou estourar o limite de rodadas, este nó força `make_move` via `tool_choice`.
5. **fallback:** depois de falhas repetidas, usa a melhor jogada do Alfa-Beta e registra o evento.
6. **memorize (memória):** o plano declarado em `make_move` vira o "plano anterior" do turno seguinte. A justificativa fica gravada no histórico da partida, que também é consultável por `get_move_history`.

**Memória.** O histórico completo da partida (`MoveRecord`: lance, jogador, peças viradas, placar, justificativa) fica no ambiente. A memória do agente acrescenta o plano corrente. Para economizar tokens, cada turno começa com uma lista nova de mensagens, e a memória entra como um **resumo compacto** em vez de repetir toda a conversa anterior.

**Explicabilidade.** Toda jogada dos agentes LLM vem acompanhada de uma justificativa em português e de um plano. A interface mostra os dois em tempo real.

### 5.5 Engenharia de custo

No plano gratuito do Groq, cada modelo tem limite de **8 mil tokens por minuto** e um teto diário. A primeira versão do Planejador gastava cerca de 4,2 mil tokens por lance, em 4 chamadas (analisar → responder → decidir → executar). Duas mudanças baixaram isso para cerca de 3,1 mil tokens e 2,5 chamadas por lance:

- `make_move` passou a ficar disponível já no nó `think`;
- o nó `decide` virou apenas uma rede de segurança.

Também foram adicionados:

- um limitador de requisições compartilhado entre os agentes (`InMemoryRateLimiter`);
- novas tentativas com *backoff* para erros 429;
- `reasoning_effort=low` nos modelos gpt-oss;
- interrupção limpa do experimento quando a cota diária acaba. A partida incompleta é descartada e pode ser retomada com `--start`.

## 6. Interface e modos de jogo

A interface em **Pygame** (`ui/pygame_app.py`) tem um menu para escolher o jogador de cada cor e atalhos para **Humano × Agente**, **Agente × Agente** e clássicos. Durante a partida ela mostra:

- o tabuleiro, com as jogadas válidas do humano marcadas, o último lance destacado e as peças viradas indicadas;
- o placar e de quem é a vez, com um aviso “pensando…” e o tempo decorrido;
- para cada lado: a jogada, a **justificativa**, o **plano**, as ferramentas chamadas, o tempo, o nº de chamadas à LLM e os tokens;
- o histórico de lances.

Os agentes rodam numa *thread* separada, para a janela não travar enquanto a LLM pensa. Controles: ESPAÇO pausa, `+`/`-` ajustam o atraso entre lances, R reinicia, ESC volta ao menu. Também há um modo texto (`python main.py --cli`).

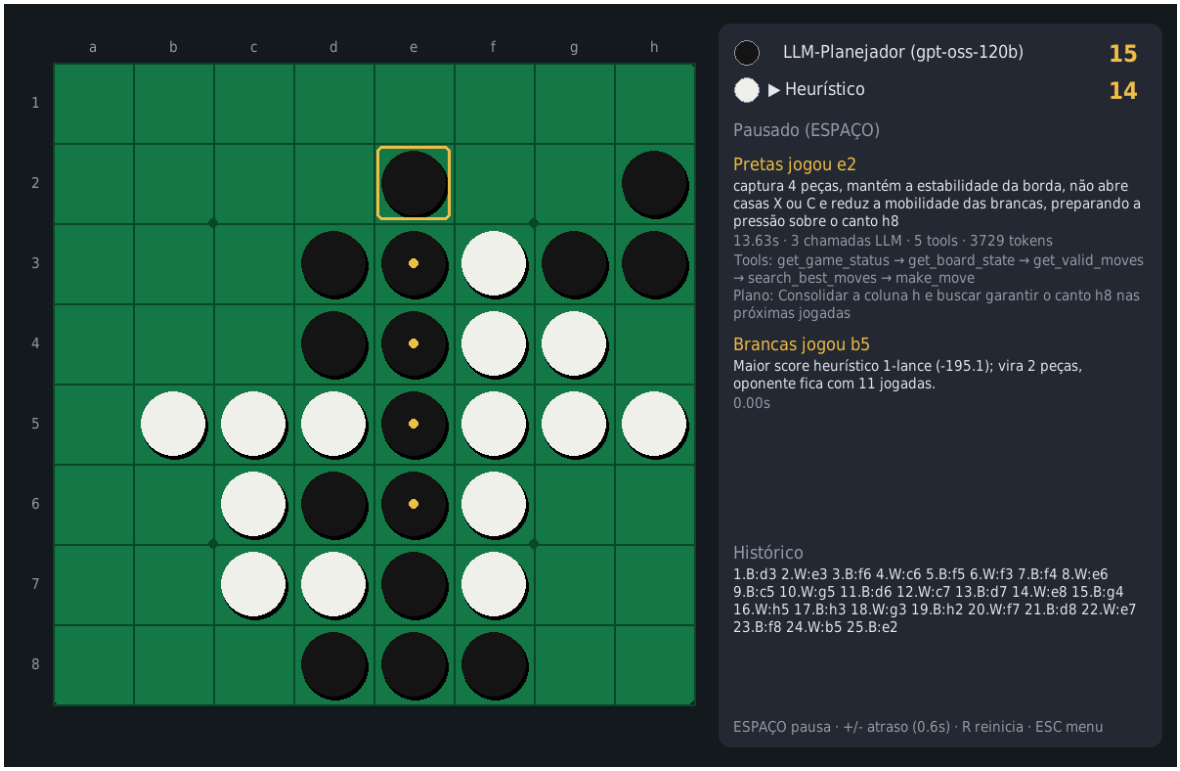



Figura 3 — Interface no lance 25 da partida LLM Planejador (pretas) × Heurístico, reproduzindo dados registrados no experimento. O painel mostra a justificativa, o plano, as ferramentas chamadas e o custo do lance. A menção ao canto h8 (distante de e2) é um exemplo do erro de orientação espacial discutido na seção 8.

## 7. Experimentos

**Metodologia:**

- cada confronto alterna as cores: as partidas 2k e 2k+1 usam **a mesma abertura** sorteada (4 lances aleatórios), com as cores trocadas. Isso diversifica as partidas (sem a abertura, dois jogadores determinísticos repetiriam o mesmo jogo) e anula a vantagem de começar;
- os resultados são gravados lance a lance em `results/turns.csv` e por partida em `results/games.csv`;
- a análise (`experiments/analyze.py`) gera as tabelas e os gráficos.

**Limitação de orçamento.** Os agentes LLM usam o plano gratuito do Groq. Uma partida do Planejador consome cerca de 90 mil tokens e leva ~8 min (o limite de 8 mil tokens por minuto domina o tempo). Por isso:

- os confrontos sem LLM têm 50 partidas cada;
- os confrontos com LLM têm poucas partidas (indicado em cada tabela);
- a comparação direta Básico × Planejador usou `gpt-oss-20b`, que tem cota própria, com o **mesmo modelo nos dois lados** para a comparação ser justa;
- as partidas contra o Heurístico usaram `gpt-oss-120b`.

# 8. Resultados e análise

## 8.1 Linhas de base (sem LLM)

| Confronto | Partidas | Resultado | Saldo médio de peças | Tempo médio/partida |
|---|---|---|---|---|
| Heurístico × Aleatório | 50 | 49 V / 0 E / 1 D (98%) | +38,2 | 0,02 s |
| Minimax-d4 × Aleatório | 50 | 50 V (100%) | +50,7 | 0,51 s |
| Minimax-d4 × Heurístico | 50 | 50 V (100%) | +48,3 | 0,60 s |

No **torneio** todos-contra-todos (5 aberturas × 2 cores por confronto; vitória 3, empate 1, derrota 0), o resultado foi: **Minimax 60 pts** (20 V, saldo +1012), Heurístico 24 pts (8 V, saldo −200) e Aleatório 6 pts (2 V, saldo −812). A hierarquia esperada se confirma, e ela valida a heurística e a busca que o agente Planejador usa como ferramentas.

## 8.2 Agentes LLM: resultados das partidas

| Confronto | Modelo | Partidas | Resultado (placares) |
|---|---|---|---|
| LLM Planejador × Heurístico | gpt-oss-120b | 1 | **Planejador venceu** (45 × 19) |
| LLM Básico × Heurístico | gpt-oss-120b | 3 | **Básico perdeu as 3** (1 × 62, 24 × 40, 5 × 57; saldo médio −43) |
| LLM Básico × LLM Planejador | gpt-oss-20b | 1 | **Planejador venceu** (59 × 5) |

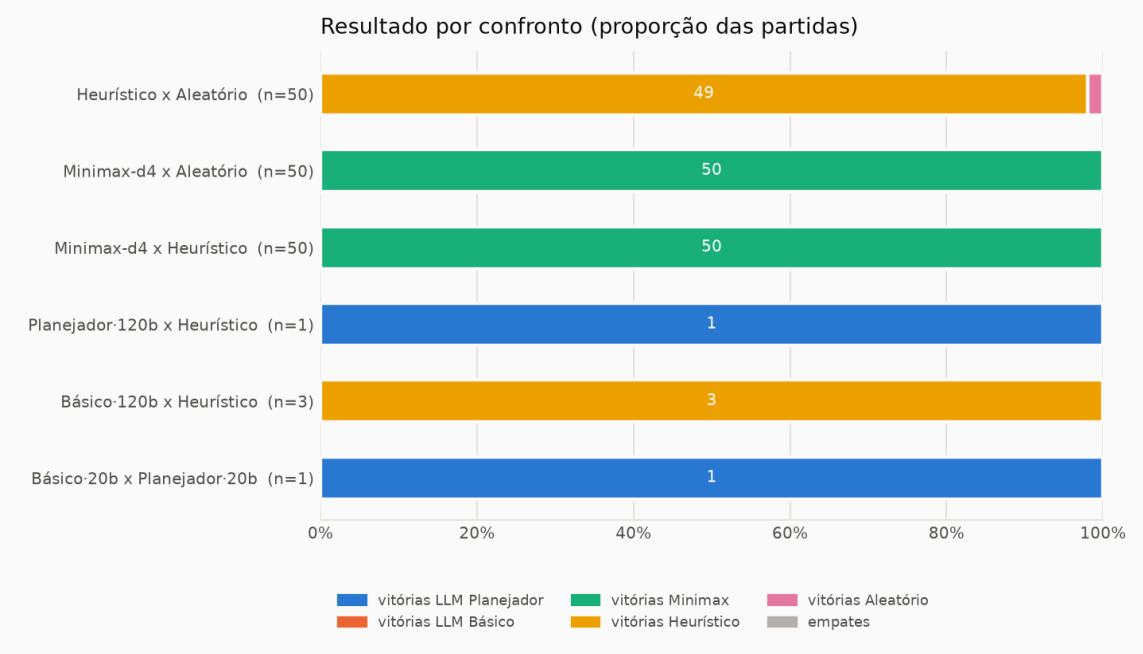



Figura 4 — Resultado por confronto. Nos confrontos com LLM o número de partidas é pequeno (cota gratuita da API).

Com tão poucas partidas não é possível estimar taxas de vitória com confiança. Mesmo assim, a direção e o tamanho das diferenças são consistentes:

- o **Básico perdeu todas** as partidas para o Heurístico, um adversário de 1 lance de profundidade;
- o **Planejador venceu todas**, inclusive o Básico com o mesmo modelo por 59 × 5.

Por isso a análise a seguir usa o **lance** como unidade, o que dá dezenas de observações por agente.

### 8.3 Qualidade dos lances

Cada partida com LLM foi reproduzida, e cada lance (após a abertura sorteada) foi comparado com o ranking do Alfa-Beta de profundidade 4, com final exato.

| Agente | Lances | Joga o top-1 da busca | Rank médio | Canto tomado quando disponível | Casa X com canto vazio | Entrega canto |
|---|---|---|---|---|---|---|
| Planejador · 120b | 29 | **83%** | 1,31 | 4/4 | 0% | 0% |
| Planejador · 20b | 30 | **100%** | 1,00 | 4/4 | 0% | 0% |
| Heurístico | 118 | 53% | 2,22 | 11/12 | 3% | 5% |
| Básico · 20b | 26 | 54% | 2,35 | — (0 ocasiões) | 8% | 15% |
| Básico · 120b | 74 | **41%** | 2,93 | 1/2 | **8%** | **16%** |

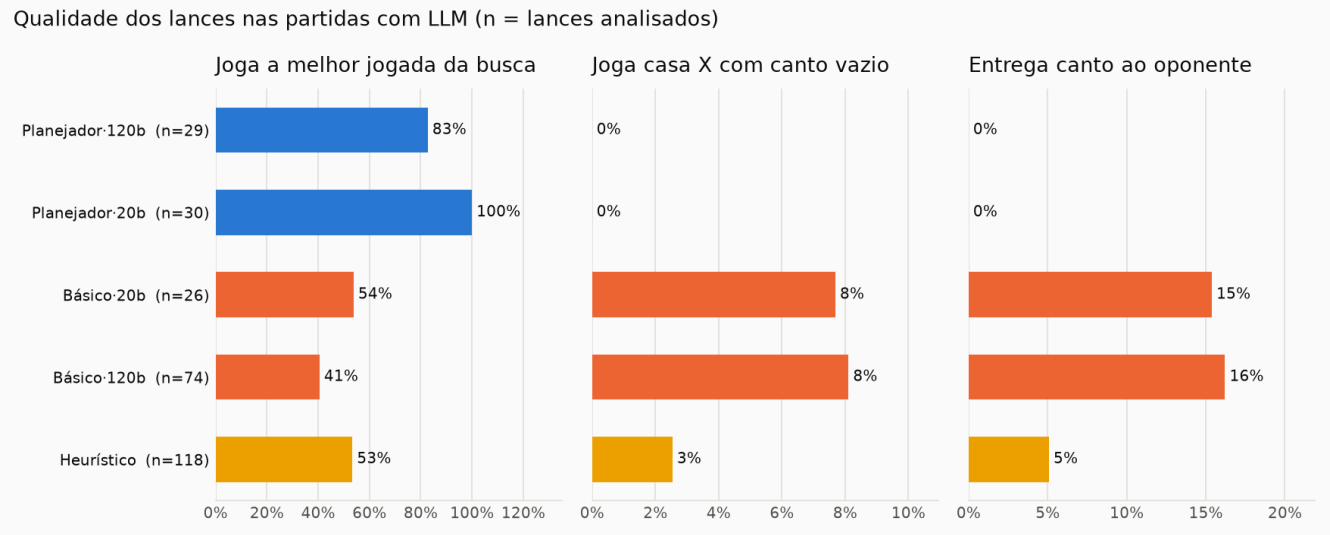



Figura 5 — Qualidade dos lances nas partidas com LLM (n = lances analisados).

O LLM Básico joga como um iniciante. Em 16% dos lances ele **entrega um canto** ao oponente e em 8% joga numa **casa X** com o canto vazio, os dois erros mais conhecidos do Othello. O Planejador, com as ferramentas de análise, não cometeu nenhum dos dois.

### 8.4 O que a LLM faz quando discorda da busca?

Esta análise isola a contribuição da LLM no Planejador. Nos lances em que ela **não** seguiu o top-1 do Alfa-Beta, medimos a perda de score em relação à melhor jogada.

| Agente | Divergências | Perda mediana | Quase equivalentes (perda < 15) | Graves (perda ≥ 100) |
|---|---|---|---|---|
| Planejador · 120b | 5/29 (17%) | 40 | 2 | 2 |
| Planejador · 20b | 0/30 (0%) | — | — | — |
| Heurístico | 55/118 (47%) | 28 | 18 | 12 |
| Básico · 20b | 12/26 (46%) | 80 | 2 | 5 |
| Básico · 120b | 44/74 (59%) | 85 | 4 | 16 |

Exemplos de divergências do Planejador · 120b:

- lance 29: **e1** em vez de g8 (perda 5): “Jogada segura na borda inferior, captura 1 peça, não abre casas X/C…”. A escolha é defensável e quase equivalente, mas e1 fica na borda **superior**. Esta partida foi jogada antes da correção da legenda de orientação;
- lance 31: **a5** em vez de g8 (perda 108): “Jogada na borda segura, captura 3 peças…”;
- lance 49 (final): **g6** em vez de b1 (perda 503): “G6 vira 9 peças, aumenta vantagem…”. Aqui a LLM cedeu à **gula por peças**, que o próprio prompt desaconselhava.

**Leitura.** O modelo maior “pensa por conta própria” em 17% dos lances. Em parte desses casos a divergência é inofensiva, mas a mediana é negativa e há duas divergências graves. O modelo menor delegou integralmente à busca. Nos dois casos, **a força do Planejador vem da ferramenta de planejamento**. A contribuição da LLM é orquestrar, ou seja, decidir o que consultar e combinar as análises, e explicar as decisões. Divergir da busca, em média, não acrescentou força.

### 8.5 Uso de ferramentas e custo

| Agente | Chamadas LLM/lance | Tool calls/lance | Tokens/lance | Latência/lance | Jogadas inválidas | Fallbacks |
|---|---|---|---|---|---|---|
| Planejador · 120b | 2,55 | 5,28 | 3 119 | 17,3 s | 0 | 0 |
| Planejador · 20b | 3,20 | 5,57 | 4 038 | 27,1 s | 0 | 0 |
| Básico · 120b | 2,73 | 2,73 | 1 717 | 8,9 s | 8 (todas corrigidas) | 0 |
| Básico · 20b | 2,96 | 2,65 | 2 112 | 14,5 s | 0 | 0 |
| Minimax-d4 | — | — | — | 0,02 s | — | — |

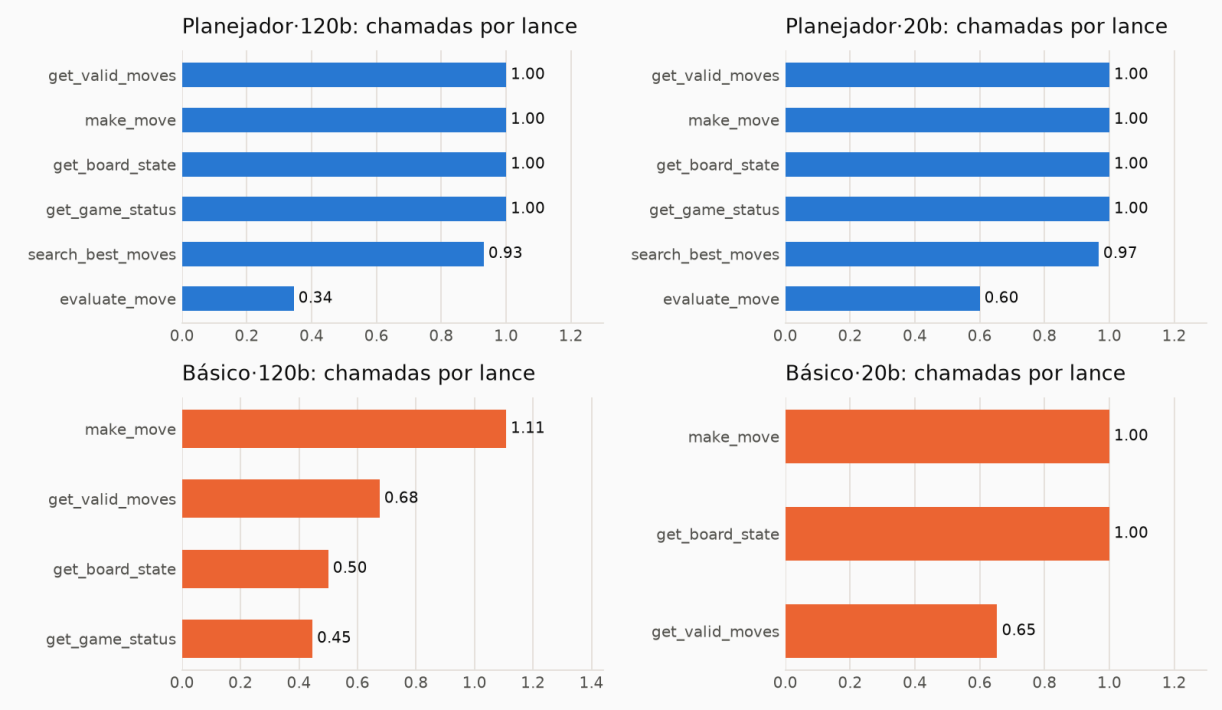



Figura 6 — Ferramentas chamadas por lance. No Planejador, get_game_status, get_board_state e get_valid_moves são chamadas pelo nó *observe*; search_best_moves e evaluate_move são escolhidas pela LLM.

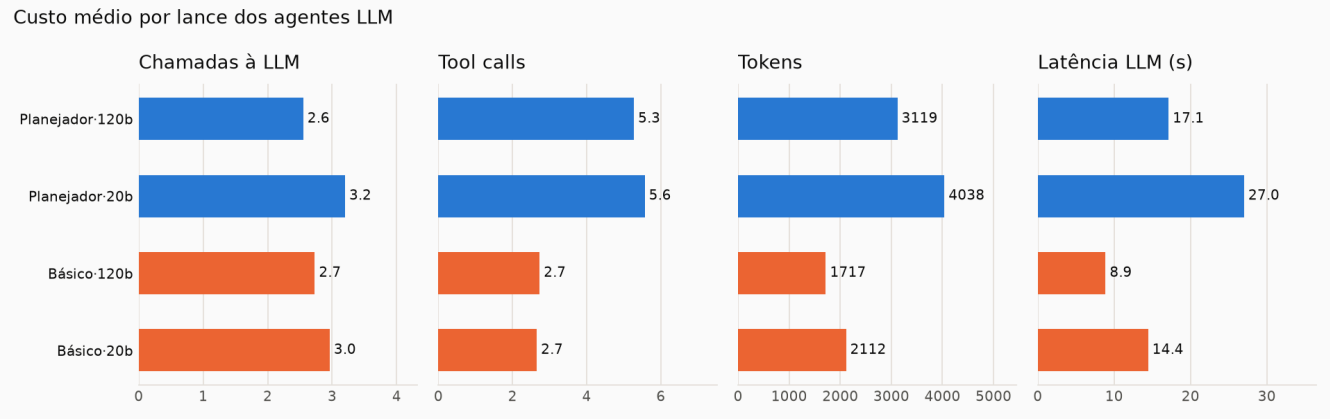



Figura 7 — Custo médio por lance dos agentes LLM.

- **Hábitos de observação do Básico.** Livre para escolher, o Básico · 120b consultou `get_valid_moves` em só 68% dos lances e `get_board_state` em só 50%. Em vários turnos ele jogou "às cegas", a partir do histórico. Isso explica as 8 tentativas de jogada inválida. Todas foram corrigidas pelo **feedback do ambiente**: o erro de `make_move` traz a lista de jogadas válidas.
- **O Planejador escolheu bem as ferramentas.** Usou `search_best_moves` em 93–97% dos lances e `evaluate_move` em 34–60%. Nunca usou `get_move_history`, porque a memória já chega resumida na observação.
- **Custo.** O Planejador gasta cerca de 2× mais tokens que o Básico e é três ordens de grandeza mais lento que o Minimax puro. Uma partida do Planejador consome ~90 mil tokens (120b), quase metade da cota diária gratuita.
- **Robustez.** O gpt-oss-20b gerou chamadas malformadas (nome de ferramenta corrompido, erro 400 `tool_use_failed`). A repetição automática da chamada resolveu todos os casos, e não houve **nenhum fallback** em todas as partidas com LLM.

### 8.6 Análise qualitativa das justificativas

- **Boas explicações, alinhadas à estratégia:** “Captura canto a8, reduz mobilidade do oponente e mantém estabilidade” (Planejador · 20b). “c5 vira apenas 1 peça, evita casas X/C e não entrega cantos; mantém mobilidade razoável na abertura” (Planejador · 120b).
- **Erros típicos do Básico:** “captura duas peças e aproxima um canto” ao jogar **b2** (casa X, entrega o canto a1; perda 250). “Ganha 2 peças e fortalece a coluna g” ao jogar **g2** (casa X; perda 244). O raciocínio é guloso e local, sem visão do que o oponente pode responder.
- **A explicação nem sempre é fiel à decisão:** “b6 maximiza mobilidade **do oponente**” (Planejador · 20b). A jogada era a melhor da busca, mas a justificativa descreve o contrário do objetivo. As justificativas ajudam a entender o agente, mas não são uma prova do raciocínio.
- **Orientação espacial:** mesmo com a legenda explícita, o 20b ainda descreveu d6 como “borda superior”. Planos como “buscar captura de h8” aparecem em lances sem relação com esse canto. Por isso as relações geométricas (casa X/C, canto entregue) foram calculadas por `evaluate_move` em vez de deixadas para a LLM.
- **Memória e plano:** o Planejador mantém planos coerentes entre turnos, como “Manter bordas estáveis e buscar controle de colunas”. O plano de um turno aparece como “plano anterior” no seguinte.

## 9. Dificuldades encontradas e como foram tratadas

| # | Dificuldade | Impacto | Como foi tratada |
|---|---|---|---|
| 1 | O modelo planejado inicialmente (`llama-3.3-70b-versatile`) foi **descontinuado** pelo Groq | Troca de modelo no meio do desenvolvimento | Adoção do `openai/gpt-oss-120b`, com o modelo configurável por `.env` / `--model` e `reasoning_effort=low` para conter os tokens de raciocínio |
| 2 | **Limites do plano gratuito**: 8 mil tokens/min, 1 000 requisições/dia e **200 mil tokens/dia por modelo** (confirmado pela própria mensagem de erro 429 da API) | Uma partida do Planejador (~90 mil tokens) consome quase metade da cota diária, e o limite por minuto força ~15–27 s por lance | Otimização do grafo (de 4,2 mil para 3,1 mil tokens e de 4 para 2,5 chamadas por lance), limitador de requisições compartilhado e novas tentativas com *backoff* |
| 3 | **Cota diária esgotada no meio de um experimento** | Partidas incompletas ou decididas por fallback contaminariam os resultados | Detecção do erro de cota diária (`DailyLimitError`): a partida em andamento é descartada e o experimento para, indicando como retomar (`--start N`) |
| 4 | **Prazo de entrega no mesmo dia** dos experimentos | Não foi possível espalhar as partidas por vários dias | Execução em paralelo em dois modelos com cotas independentes (120b e 20b), com o mesmo modelo nos dois lados no confronto direto. Análise por lance (dezenas de observações por agente) e amostra pequena declarada explicitamente |
| 5 | **Chamadas de ferramenta malformadas** no `gpt-oss-20b` (nome `make_move<\|channel\|>commentary`, erro 400 `tool_use_failed`) | Na primeira tentativa o Básico caiu no fallback aleatório, o que distorceria a comparação | Execução interrompida e descartada. A chamada passou a ser repetida automaticamente (até 3 vezes), e não houve nenhum fallback depois disso |
| 6 | **Erros de orientação espacial da LLM** (“borda inferior” para a linha 1; cantos citados sem relação com a casa jogada) | Justificativas erradas e risco de decisões ruins | Legenda explícita no tabuleiro (linha 1 = borda de cima) e relações geométricas calculadas por `evaluate_move`. O problema foi **reduzido, mas não eliminado** |
| 7 | **Justificativas nem sempre fiéis** à jogada (ex.: “maximiza mobilidade do oponente”) | A explicação pode enganar quem acompanha a partida | Registrado como limitação; métricas objetivas por lance (concordância com a busca, casas X, cantos entregues) complementam as justificativas |
| 8 | **Cotas do 120b e do 20b esgotadas** antes da gravação do vídeo | As partidas com LLM não rodariam na demonstração | No vídeo, os agentes LLM usam o `qwen/qwen3.8-27b` (cota própria), validado antes num lance de cada agente. **Os resultados deste relatório são apenas dos modelos gpt-oss.** |
| 9 | Jogadores determinísticos **repetiam a mesma partida** | Experimentos clássicos sem variação | Aberturas aleatórias (4 lances) com cores alternadas na mesma abertura |
| 10 | **Escala desbalanceada na heurística** (o termo de estabilidade dominava, com scores de cerca de −1900) | Avaliações distorcidas | Estabilidade passou a ser a diferença absoluta de peças estáveis, revalidada com os confrontos clássicos |
| 11 | **Interface travada** enquanto a LLM pensa (10–30 s) | Janela sem resposta | Agentes executados numa *thread* separada, com indicador “pensando…” e tempo decorrido |

## 10. Limitações

- **Amostra pequena nos confrontos com LLM**, por causa da cota gratuita da API. As taxas de vitória desses confrontos são indicativas, não estatisticamente conclusivas.
- **O Planejador depende das ferramentas para ser forte.** Grande parte da qualidade vem da busca Alfa-Beta exposta como ferramenta. A contribuição da LLM é escolher as ferramentas, combinar critérios, explicar e, às vezes, divergir da busca.
- **Raciocínio espacial da LLM.** A LLM confunde "borda de cima" com "borda de baixo" e cita cantos que não têm relação com a casa jogada. A legenda explícita de orientação **reduziu, mas não eliminou** o problema (seção 8.6). Por isso as relações geométricas que importam para a decisão (casa X/C, canto entregue) são calculadas por `evaluate_move`.
- **Justificativas nem sempre fiéis.** A explicação em linguagem natural pode contradizer a própria jogada. Ela é útil para acompanhar o agente, mas não deve ser lida como rastreamento exato do raciocínio.
- **Latência e custo.** 10–20 s por lance contra milissegundos do Minimax, o que torna a LLM inviável como "motor de busca". O valor dela está na orquestração e na explicação.
- **Heurística com pesos ajustados à mão**, sem otimização automática (por exemplo, *self-play*).

## 11. Conclusão

Foi implementado um agente que **percebe, raciocina, usa ferramentas e age** no jogo Othello, estruturado como um grafo LangGraph com memória, validação por feedback do ambiente e mecanismos de segurança. Os experimentos levam a três conclusões principais:

1. **Uma LLM sozinha, apenas com acesso ao ambiente, joga mal.** O LLM Básico perdeu todas as partidas para um heurístico de 1 lance. Ele entregou cantos em 16% dos lances e jogou em casas X, com justificativas gulosas ("captura duas peças e aproxima um canto").
2. **As ferramentas certas transformam o agente.** Com `search_best_moves` (Alfa-Beta) e `evaluate_move`, o Planejador venceu todas as suas partidas, inclusive o Básico com o mesmo modelo por 59 × 5. Ele não entregou nenhum canto e não jogou nenhuma casa X.
3. **A força vem do planejamento; o valor da LLM está na orquestração e na explicação.** Quando o Planejador diverge da busca (17% dos lances no 120b, 0% no 20b), a divergência tende a custar pontos. O que a LLM acrescenta é decidir quais análises fazer, integrar critérios, manter um plano entre turnos, corrigir jogadas inválidas a partir do feedback e **explicar cada decisão** em linguagem natural. As explicações, porém, nem sempre são fiéis à decisão.

O custo dessa arquitetura é alto: 2,5–3,2 chamadas à LLM, ~3–4 mil tokens e ~17–27 s por lance, contra 0,02 s do Minimax. Como trabalhos futuros, destacam-se:

- mais partidas com LLM, com cota paga, para estimar taxas de vitória com intervalos de confiança;

- chamar a LLM apenas nos lances em que a busca indica jogadas próximas, reduzindo o custo;
- dar à LLM ferramentas de busca mais profundas ou um ajuste dos pesos heurísticos por *self-play*;
- participar do torneio entre grupos usando a interface comum de jogadores já implementada.

## Apêndice — Como executar

```
pip install -r requirements.txt && cp .env.example .env   # colocar GROQ_API_KEY
python main.py                                               # interface gráfica
python main.py --black humano --white llm-planejador --start
python -m experiments.run_experiments --pair llm-basico:llm-planejador --games 4
python -m experiments.tournament --players aleatorio heuristico minimax
python -m experiments.analyze
pytest -q
```